OverlapAdvisor: Modeling and Adapting Computation–Communication Overlap for Distributed MoE Serving

Chutong Ding
Shanghai Jiao Tong University
China
dingct2004@sjtu.edu.cn

Kaixuan Zhang
Shanghai Jiao Tong University
China
zks1anx@sjtu.edu.cn

Luping Wang
Alibaba Group
China
chamu.wlp@alibaba-inc.com

Shiyou Qian
Shanghai Jiao Tong University
China
qshiyou@sjtu.edu.cn

Jian Cao
Shanghai Jiaotong University
China
cao-jian@sjtu.edu.cn

Cheng Huang
Alibaba Group
China
xiaoluo.hc@alibaba-inc.com

Guodong Yang
Alibaba Group
China
luren.ygd@taobao.com

Liping Zhang
Alibaba Group
China
liping.z@alibaba-inc.com

arXiv:2611.11857v1 [cs.DC] 9 Nov 2026

Abstract

Mixture-of-Experts (MoE) inference with expert parallelism (EP) relies on All-to-All communication, making computation–communication overlap strategies critical for achieving high performance. However, no strategy is universally optimal. Across 103 production configurations, varying input lengths, and four hardware platforms, we observe that the fastest strategy shifts based on model shape, EP degree, input length, and hardware. Since no strategy dominates more than 80.6% of configurations, configuration-aware selection is necessary. In this work, we propose OverlapAdvisor, which models overlap stages using two granularities to capture computation–communication dependencies. OverlapAdvisor builds a unified performance model, accounting for deviations from peak compute, memory, and communication performance across workloads and hardware. This model enables launch-time, configuration-aware strategy selection within two serving engines. We conduct experiments on four MoE models (30B–235B) against three baselines, utilizing SGLang and vLLM. At the kernel level, OverlapAdvisor achieves **1.16×** average and **1.59×** maximum speedups, with a prediction MAPE of **3.3%–15.9%**. In SGLang and vLLM, it improves end-to-end throughput up to **31.7%**. Its granularity-guided optimizations further achieve median speedups of **1.10×** for DeepEP-v1 and **1.22×** for FlashMoE.

1 Introduction

Trillion-parameter LLMs [13] exceed single-GPU memory and compute limits, necessitating distributed inference [15, 27, 37]. Mixture-of-Experts (MoE) is a dominant architecture for scaling capacity at fixed compute cost, with expert parallelism (EP) [16] as its most common distribution strategy. EP routes tokens to experts via All-to-All communication, whose cost scales with token count, hidden dimension, and parallelism degree, creating a critical inference bottleneck [17, 38]. Computation–communication overlap is the key to mitigating this bottleneck [38].

Many MoE EP strategies use kernel fusion and device-side scheduling for fine-grained communication–computation overlap, reducing launch and synchronization overheads. For example, Comet [33] manages dependencies for fine-grained overlap, while FlashMoE [3] fuses overlap stages using device-initiated primitives. Section 2.2 reviews these and other strategies.

Despite these advances, strategies are typically evaluated on limited model configurations and hardware, leaving no systematic comparison of major strategies under unified conditions. Prior work also implicitly assumes a universally optimal strategy, yet our measurements reveal distinct advantage regions across model shapes, input lengths, EP degrees, and hardware, with no strategy consistently dominating. Context-aware strategy adaptation is thus essential, but existing performance models characterize overlap in strategy-specific ways, preventing a common analytical comparison. Practitioners deploying MoE in SGLang [37] or vLLM [15] must therefore rely on ad-hoc benchmarking or a fixed default strategy.

We propose OverlapAdvisor, a unified model for launch-time strategy adaptation, addressing three challenges. (i) Strategies differ in kernel structure (separate kernels or a megakernel), communication/memory mechanisms (copy engine, TMA, LD/ST), and code implementations (CUTLASS, cuBLASDx, Triton, PTX), making accurate ranking under

one unified model difficult. (ii) Achieved bandwidth and throughput deviate substantially from hardware peaks; these deviations vary across the token-count range from decode to prefill and grow with finer work partitioning. (iii) Adaptation must integrate easily into serving engines and cost far less than benchmarking candidates.

Rather than introducing another overlap implementation, OverlapAdvisor treats overlap strategy adaptation as an optimization problem. It models why strategies excel in different regimes to recommend the optimal strategy for any deployment configuration—MoE shape, input length, and EP degree. To achieve this objective, OverlapAdvisor builds a unified performance model that enables launch-time strategy selection. This model can also analyze the performance bottleneck of existing strategies to suggest optimization solutions. Furthermore, OverlapAdvisor is lightweight and non-intrusive, taking about 6 µs without online profiling.

We evaluate OverlapAdvisor against six fixed-strategy baselines, 103 production MoE shapes, 10 token counts (from decode to 16K-token chunked prefill), and four GPU platforms spanning three generations: A100, H20, H800, and GB200. (1) Across 20 strategy–hardware pairs, the performance model achieves 3.3%–15.9% MAPE, and the selector achieves 85.3%–91.6% top-1 accuracy (88.43% on average). (2) OverlapAdvisor delivers 1.16× average and 1.59× maximum kernel speedups over each platform's strongest fixed strategy, and up to 5.62× over baselines. Even where MegaMoE [34] is often optimal, OverlapAdvisor outperforms it by over 5% in up to 23.83% of configurations. (3) In SGLang and vLLM, OverlapAdvisor improves end-to-end throughput over MegaMoE, UniEP [39], and DeepEP [36] by 1.78%, 4.57%, and 9.97% on average, respectively, with gains up to 31.72%. (4) OverlapAdvisor diagnoses granularity bottlenecks and guides finer-grained Dispatch in DeepEP-v1 and coarser-grained Combine in FlashMoE, achieving median speedups of 1.10× and 1.22×, respectively.

We make the following contributions:

- **A comprehensive evaluation of MoE EP overlap strategies.** We compare six strategies, including Comet [33], FlashMoE [3], Torch [22], MegaMoE [34], UniEP [39], and DeepEP [36], under identical conditions across 103 production MoE shapes, EP 2–8, 10 input lengths from decode to 16K prefill, and four platforms spanning three GPU generations.
- **A unified performance model.** Building on existing strategies, we propose a unified performance model that combines granularity-aware analytical modeling for core dependencies with data-driven efficiency modeling for specific configurations.
- **Implementation and evaluation in mainstream engines.** Implementing OverlapAdvisor within SGLang and vLLM yields significant kernel and end-to-end performance improvements. We further leverage OverlapAdvisor to analyze and optimize existing strategies.

2 Background

2.1 MoE and EP

An MoE layer replaces dense feed-forward networks with multiple experts, activating only the top-k per token to scale capacity without increasing compute cost. EP [16] distributes these experts across GPUs. Dispatch routes tokens to experts for computation, while Combine returns outputs to their origins. Both rely on All-to-All communication, whose cost depends on token count, hidden dimension, and EP degree, often creating a bottleneck in MoE inference [17, 38].

2.2 Overlap Strategies for MoE EP

Computation–communication overlap enables concurrent execution and data transfer across streaming multiprocessors (SMs), copy engines, or dedicated warps. An *overlap strategy* partitions and schedules MoE operations to exploit this concurrency. These strategies integrate communication and computation via device-side scheduling to minimize launch and synchronization overhead. For example, Comet [33] resolves dependencies for fine-grained overlap; FlashMoE [3] utilizes device-initiated primitives to fuse overlap stages; and UniEP [39] unifies strategies through parameterized MegaKernel abstractions. We evaluate Comet, FlashMoE, MegaMoE, and UniEP as overlap strategies, while using Torch- and DeepEP-based implementations as serialized baselines with sequential communication and computation. The DeepEP-based baseline uses DeepEP for communication and DeepGEMM [35] for computation.

Strategies differ in kernel partitioning, execution mechanisms, and code implementations. (1) *Communication mechanisms* transfer data across ranks via load/store pipeline (LD/ST), Tensor Memory Accelerator (TMA), or copy engines (CEs). (2) *Memory mechanisms* load data from local high bandwidth memory (HBM) via LD/ST or TMA. Both communication and memory loading use LD/ST, but with different instructions and transfer semantics: the former accesses peer memory via `ld.global`/`st.global`, while the latter uses `cp.async` for asynchronous local HBM-to-shared-memory transfers that bypass registers. (3) *Computation mechanisms* use Tensor Cores for expert computation and CUDA Cores for activations and other element-wise operations. (4) *Implementations* realize these mechanisms using low-level instructions (CUDA C++ with inline PTX), GPU DSLs (Triton), or CUDA libraries (CUTLASS and cuBLASDx).

Mechanism choices and their code implementations affect instruction scheduling and resource efficiency, yielding performance differences among similar strategies. Table 1 summarizes the execution mechanisms and implementations of the evaluated strategies.

Table 1. Execution mechanisms and implementations of MoE EP strategies.

Strategy	Comm		Mem	Comp	Impl
	Dispatch	Combine			
Comet	CE	LD/ST	LD/ST, TMA	Tensor Core	CUTLASS
FlashMoE	LD/ST	LD/ST	LD/ST	Tensor Core	cuBLASDx
UniEP	TMA	LD/ST	TMA	Tensor Core	Triton
MegaMoE	TMA	LD/ST	TMA	Tensor Core	CUDA C++
DeepEP-v1	LD/ST	LD/ST	TMA	Tensor Core	CUDA C++
DeepEP-v2	TMA	TMA			

2.3 Strategy Selection and Modeling

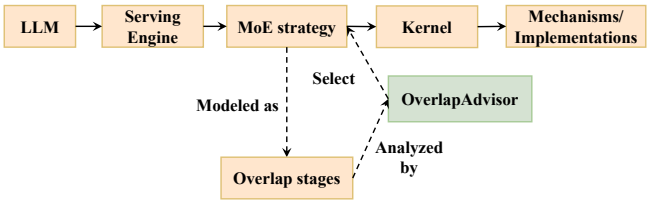

Figure 1. OverlapAdvisor in the execution stack. Solid arrows show execution flow; dashed arrows show OverlapAdvisor's optimal strategy selection via two-stage overlap analysis.

Figure 1 locates OverlapAdvisor in the execution stack: a serving engine runs an LLM whose MoE layers use EP strategies implemented by kernels with specific execution mechanisms and implementations. Given the router's assignments, OverlapAdvisor models each strategy's full *MoE layer* from Dispatch to Combine through two logical overlap stages in the execution pipeline: Dispatch+GroupGEMM couples dispatch with the first GroupGEMM, while GroupGEMM+Combine couples the second GroupGEMM with Combine. These stages are analytical units, not kernel boundaries: a strategy may split a stage across multiple kernels or fuse both stages into one kernel. OverlapAdvisor uses this stage-based analysis to predict each strategy's MoE execution time and select the fastest strategy.

2.4 Key Notations and Workload Configuration

Table 2 lists the key notations used in this paper, covering workload dimensions, hardware capabilities, kernel settings, and performance-model quantities. We define a *workload configuration* as the tuple $(H, I, E, k, S_{R_{\text{in}}}, D)$, capturing model dimensions, token counts, and parallelism settings, independent of hardware and strategy.

3 Observations

MoE EP strategy performance depends on various factors: the communication mechanism, token layout, resource partitioning, synchronization boundaries, and hardware characteristics such as SM count and interconnect bandwidth. Fixing these choices limits a strategy's effectiveness across

Table 2. Notations used in this work.

Symbol	Description
Workload	
H, I	Hidden size, MoE intermediate size
E	Total number of experts
k	Number of experts per token (top-k)
$S_{R_{\text{in}}}$	Input tokens per rank
D	Expert-parallelism degree
β	Bytes per element (BF16/FP8)
$S_{R_{\text{a}}} = S_{R_{\text{in}}} \cdot k$	Tokens assigned per rank (average)
$S_{R_{\text{a}}}^{\max}$	Tokens assigned to the hottest rank
$\rho = S_{R_{\text{a}}}^{\max} / S_{R_{\text{a}}}$	Imbalance factor
$E_{\text{L}} = E/D$	Local experts per rank
$S_E = S_{R_{\text{a}}} / E_{\text{L}}$	Tokens assigned per expert (average)
Hardware	
F	Peak compute rate (BF16/FP8)
$B_{\text{HBM}}, B_{\text{NVL}}$	Peak HBM, NVLink bandwidth
N_{SM}	Number of SMs on the GPU
Kernel	
B_M, B_N	Block M and block N in GroupGEMM
Performance Model	
T	Total MoE execution time
$T_{\text{comp}}, T_{\text{comm}}$	Computation / communication time
$G_{\text{comp}}, G_{\text{comm}}$	Computation / communication granularity

deployments, yet prior work often assumes universal optimality. Figure 2 compares six MoE EP strategies, using single-layer kernel-level benchmarks with MoE configurations from DeepSeek-V3 [8] and Qwen3 [24] on A100, H800 and GB200. The evaluation covers two settings: BF16 with a standard MLP and FP8 with a gated MLP. No single strategy is optimal across the evaluated configurations and hardware platforms; different strategies excel in different regions. From this analysis, we identify three observations.

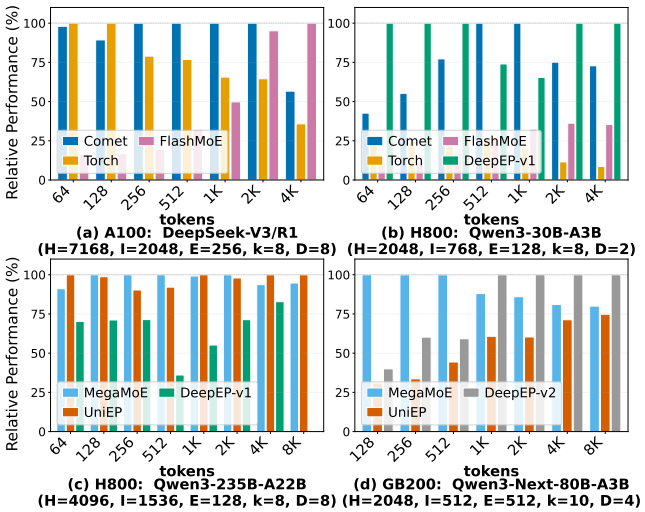

Figure 2. Relative performance of six MoE EP strategies at the kernel level, normalized per configuration to the fastest strategy (best is 100%). (a) and (b): BF16 with a standard MLP; (c) and (d): FP8 with a gated MLP.

Observation 1: Existing strategies lack general effectiveness.

Figure 2 shows that different strategies excel under different conditions, but prior evaluations cover limited configurations and may miss these trade-offs. Table 3 summarizes the evaluation scope of strategies by the number of reported $(H, I, E, k, S_{R_{\text{in}}}, D)$ configurations. We omit D as a separate column because it remains fixed for each $(H, I, E, k, S_{R_{\text{in}}})$ tuple. Existing evaluations utilize diverse, self-selected hardware and workloads. For instance, FlashMoE compares against Comet and Megatron [19] on H100 across 14 configurations, while Comet compares against Megatron and FasterMoE [31] on H800 and L20 across 22 configurations. Due to the lack of a common platform or unified configuration, these results neither facilitate direct comparison nor establish general performance metrics beyond the specific workloads evaluated.

Observation 2: Existing kernel performance models are strategy-specific, not unified.

Current kernel performance models fail to support context-aware strategy selection. Most models approximate overlap as $\max(T_{\text{comp}}, T_{\text{comm}})$, while refinements involving non-overlappable terms remain strategy-specific [26, 39]. Lacking a unified performance model for guidance, practitioners rely on costly benchmarking or potentially suboptimal defaults.

Observation 3: Effective strategies require configuration adaptation.

Achieving effective MoE EP overlap is challenging, which is manifested in three aspects. First, input-dependent routing creates dynamic communication–computation dependencies, which complicates static partitioning for fine-grained pipelining [33]. Second, mapping tokens to destination buffers introduces extra metadata and synchronization costs [40]. Third, overlap also causes contention for SM resources, memory bandwidth, and L2 cache [7, 41]. Furthermore, GPU multi-stream scheduling remains non-deterministic in real-world production environments [6]. Consequently, selecting the right strategy requires balancing both hardware and workload factors, resulting in a complex, high-dimensional optimization problem that makes manual tuning impractical.

4 Design

4.1 Overview

As illustrated in Figure 3, the overall structure of OverlapAdvisor consists of three components.

Table 3. Evaluation scope of MoE overlapping strategies.

Strategy	(H, I, E, k)	$S_{R_{\text{in}}}$	Configurations	Platforms
Comet	10	5	22	H800, L20
FlashMoE	6	3	14	H100
UniEP	12	2	24	2 Hopper clusters
OverlapAdvisor	**103**	**10**	**1030**	**A100, H20, H800, GB200**

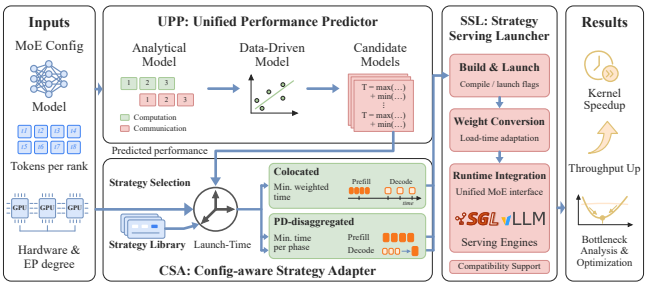

Figure 3. Overall structure of OverlapAdvisor.

A Unified Performance Predictor (UPP). To enhance both generality and accuracy, we employ a hybrid approach to model the kernel performance of both overlap and serial strategies. First, OverlapAdvisor constructs an analytical model for MoE strategies from the perspective of execution granularity. Second, we treat efficiency-related parameters that cannot be directly derived from hardware specifications, such as memory and communication bandwidth utilization and peak computational efficiency, as strategy-specific variables to be fitted. OverlapAdvisor samples the execution time of each strategy using only a limited set of configurations, and utilizes a differential evolution (DE) [25] optimizer to estimate these parameters. Finally, incorporating these fitted parameters yields hardware- and strategy-aware models that predict performance for arbitrary configurations (§4.2).

A Configuration-aware Strategy Adapter (CSA). At server launch, OverlapAdvisor reads the deployment configuration and uses UPP to select strategies that minimize predicted MoE execution time. For PD-disaggregated deployments, CSA selects prefill and decode strategies independently; for colocated deployments, it selects a shared strategy by weighting each phase's predicted time. Since all inputs are deployment invariants, selection runs once in about 6 μs, requiring no profiling or runtime adaptation (§4.3).

A Strategy Serving Launcher (SSL). Since a decision is only valuable if it can be implemented, multiple strategies are integrated into serving engines (such as SGLang [37], vLLM [15]) via a unified, interchangeable MoE backend interface. SSL applies CSA's selections through launch options or environment variables, supporting hardware-specific kernel builds, load-time weight conversion, and lightweight runtime integration. These additive changes require only minor compatibility adjustments, preserve existing code paths, and leave non-MoE layers unchanged (§4.4).

4.2 Unified Performance Predictor (UPP)

Predicting a strategy's kernel execution time requires two key components: an analytical model of the fundamental computation–communication dependencies based on a granularity abstraction, independent of strategy-specific kernel

implementations; and data-driven fitting of efficiency parameters that capture bandwidth utilization and compute efficiency while retaining a shared prediction structure across strategies. OverlapAdvisor derives the former analytically (§4.2.1) and fits the latter from a small sample set (§4.2.2).

4.2.1 Granularity-aware Analytical Modeling. We focus on Dispatch + GroupGEMM and GroupGEMM + Combine, which dominate MoE layer execution time and capture key overlap differences across strategies. The total execution time is determined by combining these two stages as follows:

$$\widehat{T} = T^{(1)} + T^{(2)} + \sum_j T_j^{\text{misc}}, \tag{1}$$

where $T^{(1)}$ and $T^{(2)}$ model Dispatch+GroupGEMM and GroupGEMM+Combine, respectively. T_j^{misc} covers memory-bound kernels (e.g., SwiGLU/GELU element-wise activations, permute that groups tokens by expert, and unpermute that restores their original order).

Modeling the two stages involves characterizing two factors: the granularity at which communication data enables computation, and the granularity at which computation results enable communication. Let T_{comp} and T_{comm} represent the computation and communication time within the Dispatch+GroupGEMM or GroupGEMM+Combine stages. Serialized execution gives $T = T_{\text{comp}} + T_{\text{comm}}$, reflecting strict data dependencies; perfect overlap gives $T = \max(T_{\text{comp}}, T_{\text{comm}})$, assuming no such dependencies. In practice, however, the execution time falls between these two bounds because computation and communication exhibit data dependencies at a specific granularity. We abstract each computation– communication stage using two granularities, G_{comp} and G_{comm}, defined as the numbers of rounds into which computation and communication are partitioned. These quantities characterize the granularity of data dependencies between computation and communication.

Overall, in each MoE EP stage, three factors impact execution time. The first is the temporal ordering of computation and communication. The second is the execution time of computation and communication. As the communication workload is $O(S_{R_{\text{in}}} \cdot H)$ while the computation workload is $O(S_{R_{\text{in}}} \cdot H \cdot I)$, the dominant term alternates between T_{comp} and T_{comm} as I varies. The third is the granularity, which partitions tasks into various rounds. Figure 4 shows stage execution times for all eight combinations of the three factors. We model each computation–communication stage as:

$$T = \max(T_{\text{comp}}, T_{\text{comm}}) + \frac{\min(T_{\text{comp}}, T_{\text{comm}})}{\min(G_{\text{comp}}, G_{\text{comm}})}, \tag{2}$$

For example, Figure 4(a) has $T_{\text{comp}} > T_{\text{comm}}$, $G_{\text{comp}} = 4$, and $G_{\text{comm}} = 2$, giving $T = T_{\text{comp}} + T_{\text{comm}}/2$.

Contention for SM resources, L2 cache, and bandwidth during overlap is captured by the data-driven model below rather than explicitly modeled in this equation.

4.2.2 Data-Driven Efficiency Modeling for Strategies and Platforms. Routing-induced load imbalance across ranks affects MoE execution time. Following [11], we quantify it as $\rho = S_{R_a}^{\max}/S_{R_a}$, where S_{R_a} and $S_{R_a}^{\max}$ denote the average and maximum numbers of tokens received per rank, respectively; $\rho = 1$ indicates uniform loading. To model inter-rank imbalance, we approximate the load of each local expert on the busiest rank by the rank's average expert load, $\rho S_E = S_{R_a}^{\max}/E_{\text{L}}$, where E_{L} is the number of local experts per rank and $S_E = S_{R_a}/E_{\text{L}}$ is the average expert load. This approximation does not explicitly capture load imbalance among experts within a rank. We evaluate all granularities at this busiest-rank load to approximate the execution bottleneck.

While the analytical model captures the overlap structure, it fails to fully characterize actual performance. Data-driven efficiency modeling improves accuracy and distinguishes between strategies with similar overlap granularities but different efficiencies, such as MegaMoE and UniEP. The two primary factors affecting resource efficiency in MoE EP are bandwidth and GEMM.

Bandwidth Efficiency Modeling. In GPU systems, the effective bandwidth of NVLink and HBM varies with the volume of data transferred [26]. At small volumes, startup overheads, such as kernel launch or communication startup, account for a large fraction of execution time, and limited parallelism further constrains bandwidth utilization. As data volume increases, these overheads are amortized, and effective bandwidth generally rises before reaching saturation. The saturation level can be below the hardware peak and depends on the platform and the data-movement mechanism, such as CE, TMA, or LD/ST instructions (Table 1).

UPP uses a shared capped power law to model communication and HBM bandwidth efficiency across strategies:

$$\eta_{\text{comm/mem}} = \min\left(\eta_{cap}, \left(\frac{V}{V_{ref}}\right)^{\alpha}\right), \tag{3}$$

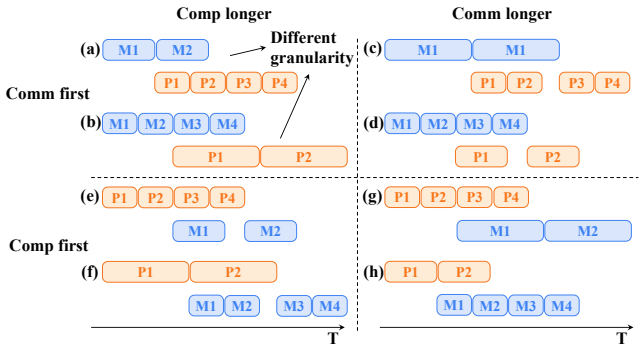

Figure 4. Overlap schematic (M: communication; P: computation), showing how T is determined by the sequence of execution–whether computation or communication takes precedence–as well as the dominant time factor and the coarser granularity between the two.

where η_{cap} is the saturation efficiency, V is the data volume, V_{ref} is a reference volume that sets the scale of the efficiency curve, and α is the power-law exponent. Below $V_{\text{sat}} = V_{\text{ref}}\eta_{\text{cap}}^{1/\alpha}$, efficiency increases with V, and transfer time grows as $V^{1-\alpha}$. Above saturation, efficiency stays at η_{cap} and transfer time grows linearly. For a fixed saturation point, larger α gives a steeper efficiency decline at smaller volumes. We fit η_{cap}, V_{ref}, and α for each strategy and platform.

We model two primary communication topologies: ring-based (Comet dispatch; Torch) and P2P (Comet combine; FlashMoE, MegaMoE, UniEP, DeepEP), as follows:

$$V_{\text{comm}} = \begin{cases} (D-1)\cdot S_{R_{\text{in}}}\cdot H\cdot\beta, & \text{ring-based,} \\ \rho\cdot(D-1)\cdot\dfrac{S_{R_{\text{in}}}}{D}\cdot k\cdot H\cdot\beta, & \text{P2P-based.} \end{cases} \tag{4}$$

$$T_{\text{comm}} = \frac{V_{\text{comm}}}{B_{\text{NVL}}\cdot\eta_{\text{comm}}(V_{\text{comm}}/G_{\text{comm}})}, \tag{5}$$

where D is the EP degree, $S_{R_{\text{in}}}$ the input tokens per rank, H the hidden size, k selected experts, β the bytes per element (e.g., BF16 or FP8), and B_{NVL} the peak NVLink bandwidth. P2P communication time is determined by the busiest rank's receive load, and η_{comm} is evaluated at the per-round volume $V_{\text{comm}}/G_{\text{comm}}$.

GroupGEMM memory-access time T_{mem} is modeled as:

$$V_{\text{mem}}^{(i)} = \begin{cases} S_{R_{\text{a}}}^{\max}(H+mI)\,\beta \;+\; E_{\text{L}}\cdot m\cdot HI\beta, & i=1, \\ S_{R_{\text{a}}}^{\max}(I+H)\,\beta \;+\; E_{\text{L}}\cdot HI\beta, & i=2, \end{cases} \tag{6}$$

$$T_{\text{mem}}^{(i)} = \frac{V_{\text{mem}}^{(i)}}{B_{\text{HBM}}\cdot\eta_{\text{mem}}\left(V_{\text{mem}}^{(i)}/G_{\text{comp}}\right)}, \tag{7}$$

where $i = 1, 2$ denote up- and down-projection GroupGEMMs, respectively, and m denotes the number of first-layer projection matrices (1 for standard MLPs, 2 for gated MLPs). $S_{R_{\text{a}}}^{\max}$ is the busiest rank's received token count, E_{L} local experts per rank, I the MoE intermediate size, and B_{HBM} the peak HBM bandwidth. Since each GroupGEMM runs in G_{comp} rounds, η_{mem} is evaluated at the per-round volume $V_{\text{mem}}^{(i)}/G_{\text{comp}}$. Small memory-bound kernels, including activation kernels, permute, and unpermute, are modeled similarly.

GEMM Efficiency Modeling. Following the Roofline model [29], GroupGEMM is limited by compute or memory performance. As shown in Table 1, all strategies use Tensor Cores but differ in memory-access mechanisms and kernel implementations. The bottleneck depends on problem size: more tokens per expert increase weight reuse and arithmetic intensity, shifting GroupGEMM toward the compute-bound regime. Conversely, when the number of tokens is small, the memory-access overhead of the weight matrices cannot be sufficiently amortized, causing the operation to be memory-bound. Consequently, in the compute-bound regime, a large number of tiles are evenly distributed across all SMs, and computational efficiency remains approximately constant at η_{gemm}. In the memory-bound regime, memory-access efficiency varies with data volume and is modeled using the same power-law efficiency formulation as Eq. (3). Here, η_{gemm} is fitted for each strategy and hardware.

We model GroupGEMM execution time T_{comp} as:

$$C_{\text{gemm}}^{(i)} = \begin{cases} 2\,m\,S_{R_{\text{a}}}^{\max}\,HI\,\gamma_{\text{pad}}, & i=1, \\ 2\,S_{R_{\text{a}}}^{\max}\,HI\,\gamma_{\text{pad}}, & i=2, \end{cases} \tag{8}$$

$$T_{\text{comp}}^{(i)} = \max\left(\frac{C_{\text{gemm}}^{(i)}}{F\cdot\eta_{\text{gemm}}},\; T_{\text{mem}}^{(i)}\right), \tag{9}$$

where C_{gemm} is the floating-point workload of GroupGEMM, F is the peak compute rate (TFLOPs), and $\gamma_{\text{pad}} = \frac{\lceil \rho S_E/B_M\rceil B_M}{\rho S_E}$ is the padding inflation factor along the token dimension, which accounts for the computational waste incurred when ρS_E is not an integer multiple of B_M.

Parameter Fitting. The performance models established above for MoE EP strategies include several physical parameters that must be fitted, primarily the bandwidth efficiency parameters (η_{cap}, V_{ref}, α) and the GEMM efficiency parameter (η_{gemm}). These parameters cannot be read off directly from hardware specifications. For each ⟨strategy, platform⟩ pair, we fit the relevant parameters separately for each kernel (e.g., Dispatch, permute, GroupGEMM, and GELU) by minimizing MAPE over n measured configurations. Parameter bounds enforce physical validity (e.g., $\alpha, \eta_{\text{gemm}} \in (0, 1)$). Since the model's min, max, and $\lceil\cdot\rceil$ operations make the objective nonsmooth and nonconvex, we use DE for global optimization rather than gradient-based methods [14, 18].

4.3 Configuration-aware Strategy Adapter (CSA)

MoE EP strategies differ in overlap behavior, execution mechanisms and implementations, with no universal winner. Therefore, an effective serving engine must select the strategy that maximizes end-to-end throughput. OverlapAdvisor performs model-driven, launch-time selection: it uses UPP to predict candidate performance from the deployment configuration and selects the best strategy before server startup, without online profiling or runtime adaptation. Strategy selection is phase-aware because prefill and decode operate at significantly different sequence lengths.

With data-parallel attention, prefill and decode have different input token counts. The decode per-kernel token count $S_{R_{\text{in}}}^{\text{dec}}$ equals the number of concurrent requests per rank, as each request contributes one token per step. The prefill per-kernel token count $S_{R_{\text{in}}}^{\text{pre}}$ is set by the per-rank chunked-prefill budget [2]. For example, 32 decode requests yield $S_{R_{\text{in}}}^{\text{dec}} = 32$, versus $S_{R_{\text{in}}}^{\text{pre}} = 4096$ for a full 4096-token chunk. These differences place the phases at different points on the efficiency curves in Eq. (3), potentially favoring different backends even for the same model and EP degree.

For each phase $\varphi \in \{\text{pre}, \text{dec}\}$, OverlapAdvisor selects

$$s^{\star}_{\varphi} = \arg\min_{s \in \mathcal{S}} \widehat{T}_{s,\varphi}\left(H, I, E, k, S^{\varphi}_{R_{\text{in}}}, D\right), \tag{10}$$

where $\mathcal{S}$ contains the strategies available on the target platform and $\widehat{T}_{s,\varphi}$ is the predicted MoE execution time. All inputs are deployment configurations fixed during server operation, so ranking is computed only once.

CSA's launch-time selection avoids three limitations: (i) fixed strategies are suboptimal as rankings vary across configurations; (ii) profiling-based autotuning benchmarks every candidate on the target cluster at each launch or configuration change, incurring profiling overhead, whereas our model is fitted once per platform; and (iii) runtime adaptation requires online strategy evaluation and may incur switching and warm-up costs. Instead, Eq. (10) evaluates $|\mathcal{S}|$ candidates in closed form using a few hundred arithmetic operations per phase, completing selection in approximately 6 μs at launch.

OverlapAdvisor supports both PD-disaggregated deployments with separate phase-specific strategies and colocated deployments sharing one backend.

PD Disaggregation. Prefill and decode run on separate GPU sets with EP degrees D_{p} and D_{d}. OverlapAdvisor applies Eq. (10) at $(D_{\text{p}}, S^{\text{pre}}_{R_{\text{in}}})$ and $(D_{\text{d}}, S^{\text{dec}}_{R_{\text{in}}})$, configuring each instance via separate flags. Unlike fixed-backend systems, this permits heterogeneous choices, such as DeepEP for prefill and MegaMoE for decode.

Colocated Deployment. A shared backend minimizes total MoE time per GPU. For L MoE layers, B concurrent requests per rank, input length ISL, and output length OSL, the instance executes $LB\,\text{ISL}/S^{\text{pre}}_{R_{\text{in}}}$ prefill and L OSL decode MoE invocations. Weighting each phase's predicted time by its invocation count gives

$$s^{\star} = \arg\min_{s \in \mathcal{S}} \frac{L \cdot B \cdot \text{ISL}}{S^{\text{pre}}_{R_{\text{in}}}} \widehat{T}_s\left(D, S^{\text{pre}}_{R_{\text{in}}}\right) + L \cdot \text{OSL}\, \widehat{T}_s\left(D, S^{\text{dec}}_{R_{\text{in}}}\right). \tag{11}$$

4.4 Strategy Serving Launcher (SSL)

To support strategy selection, we integrate candidate strategies as interchangeable MoE backends in SGLang and vLLM, selected via existing launch options or environment variables (e.g., `--moe-a2a-backend` in SGLang and `--all2all-backend` in vLLM), leaving the rest of the serving stack unchanged.

Integration comprises three steps and minor compatibility adjustments, all additive and preserving existing code paths: (S1) *Build and Launch* (≈0.03k lines per engine): expose kernels through hardware-specific compile flags and add selector launch options. (S2) *Weight Conversion*: convert checkpoints at load time to the required kernel layout in SGLang (≈0.15k lines), and adapt weights through native online block-FP8 quantization in vLLM (≈0.05k lines). (S3) *Runtime Path*: prepare activations and routing metadata in the backend's cross-rank buffer and invoke its kernels through a lightweight MoE-forward branch in SGLang (≈0.5k lines) or a wrapper at vLLM's MoE-method selection point (≈0.35k lines). Non-MoE layers remain unchanged. *Compatibility* (≈0.02k lines per engine): bypass unquantized-weight fast paths during online quantization, adapt configurations to newer model-library versions, and size communication buffers for single-node use.

The changes span ≈0.7k lines across 11 files in SGLang (∼0.1% of ∼600k Python lines) and ≈0.4k lines across 3 files in vLLM (∼0.1% of ∼320k). Once weight conversion and runtime paths are in place, strategies sharing the same buffer and weight contracts need only minor kernel-call updates, typically tens of lines.

Table 4. Instantiation of the granularity abstraction for MoE EP strategies. Tile counts (W_1, W_2) and SM allocations (N^{P}_{SM}, N^{M}_{SM}) are strategy-specific.

Dispatch + GroupGEMM		
Strategy	G_{comm}	G_{comp}
Comet	$D-1$	$\min(\lceil \rho S_E/B_M \rceil, \lceil W_1/N_{\text{SM}} \rceil)$
FlashMoE	$\lceil DE_{\text{L}}/N^{\text{M}}_{\text{SM}} \rceil$	$\lceil W_1/N^{\text{P}}_{\text{SM}} \rceil$
MegaMoE	$\lceil S_{R_{\text{in}}} k/N_{\text{SM}} \rceil$	$\min(E_{\text{L}}\lceil \rho S_E/B_M \rceil, \lceil W_1/N_{\text{SM}} \rceil)$
UniEP	$\lceil S_{R_{\text{in}}} k/N^{\text{M}}_{\text{SM}} \rceil$	$\min(E_{\text{L}}\lceil \rho S_E/B_M \rceil, \lceil W_1/N^{\text{P}}_{\text{SM}} \rceil)$
Torch/DeepEP	1	1
GroupGEMM + Combine		
Strategy	G_{comp}	G_{comm}
Comet	$\lceil W_2/N^{\text{P}}_{\text{SM}} \rceil$	N_{split}
FlashMoE	$\lceil W_2/(N_{\text{SM}}-1) \rceil$	$\lceil W_2/(N_{\text{SM}}-1) \rceil$
MegaMoE	$\lceil W_2/N_{\text{SM}} \rceil$	$\lceil W_2/N_{\text{SM}} \rceil$
UniEP	$\lceil W_2/N^{\text{P}}_{\text{SM}} \rceil$	$\lceil S^{\max}_{R_{\text{a}}}/N^{\text{M}}_{\text{SM}} \rceil$
Torch/DeepEP	1	1

5 Implementation

5.1 Strategy Granularity Instantiation for Analytical Modeling

Table 4 instantiates the two granularities defined in Section 4.2.1 for six representative MoE EP strategies: four overlap approaches (Comet [33], FlashMoE [3], MegaMoE [34], UniEP [39]) and two serialized methods (Torch [22] and DeepEP [36]). Specifically, Comet and UniEP explicitly split execution into three kernels: Dispatch+GroupGEMM, Activation, and GroupGEMM+Combine. FlashMoE and MegaMoE utilize a single fused kernel while maintaining a strict ordering between the two overlap stages. Conversely, Torch and DeepEP execute computation and communication serially with $G_{\text{comm}} = G_{\text{comp}} = 1$, representing a rigid, coarse-grained boundary between computation and communication.

We examine the granularity of the four overlapping strategies. Let N^{P}_{SM} and N^{M}_{SM} denote the numbers of SMs allocated

to computation and communication, respectively. In Dispatch+GroupGEMM, Comet runs ring All-Gather on the CE, transferring one peer's full shard per step, so $G_{\text{comm}} = D - 1$. FlashMoE distributes DE_{L} transfer tasks, one per (peer, local-expert) pair, across N_{SM}^{M} dispatch-specialized blocks, giving $G_{\text{comm}} = \lceil DE_{\text{L}}/N_{\text{SM}}^{\text{M}} \rceil$. MegaMoE and UniEP both use $S_{R_{\text{in}}}k$ token-level communication tasks. MegaMoE uses communication warps on every SM through warp specialization, with $G_{\text{comm}} = \lceil S_{R_{\text{in}}}k/N_{\text{SM}} \rceil$, whereas UniEP uses dedicated communication SMs, with $G_{\text{comm}} = \lceil S_{R_{\text{in}}}k/N_{\text{SM}}^{\text{M}} \rceil$. The first GroupGEMM processes $W_1 = E_{\text{L}}\lceil \rho S_E/B_M \rceil \lceil mI/B_N^{(1)} \rceil$ output tiles on the hottest rank. Computation requires $\lceil W_1/N_{\text{SM}} \rceil$ waves, with N_{SM} replaced by N_{SM}^{P} for FlashMoE and UniEP. However, wave counts alone can overstate overlap: a tile becomes computable only after B_M tokens of its expert arrive. Comet receives tokens for all experts at the same rate, so a single hot expert bounds the rounds by $\lceil \rho S_E/B_M \rceil$, yielding $G_{\text{comp}} = \min(\lceil \rho S_E/B_M \rceil, \lceil W_1/N_{\text{SM}} \rceil)$. MegaMoE and UniEP accumulate tokens expert by expert, relaxing this bound to $E_{\text{L}}\lceil \rho S_E/B_M \rceil$. Thus, MegaMoE has $G_{\text{comp}} = \min(E_{\text{L}}\lceil \rho S_E/B_M \rceil, \lceil W_1/N_{\text{SM}} \rceil)$, while UniEP has $G_{\text{comp}} = \min(E_{\text{L}}\lceil \rho S_E/B_M \rceil, \lceil W_1/N_{\text{SM}}^{\text{P}} \rceil)$. FlashMoE's (peer, expert) task unit imposes no additional gating bound, so $G_{\text{comp}} = \lceil W_1/N_{\text{SM}}^{\text{P}} \rceil$.

In GroupGEMM+Combine, the second GroupGEMM processes $W_2 = E_{\text{L}}\lceil \rho S_E/B_M \rceil \lceil H/B_N^{(2)} \rceil$ output tiles on the hottest rank. Both tile counts (W_1, W_2) and SM allocations (N_{SM}^{P}, N_{SM}^{M}) are strategy-specific. Comet and UniEP use N_{SM}^{P} computation SMs, giving $G_{\text{comp}} = \lceil W_2/N_{\text{SM}}^{\text{P}} \rceil$. For MegaMoE, $G_{\text{comp}} = \lceil W_2/N_{\text{SM}} \rceil$, as computation uses all SMs. FlashMoE reserves one block for an in-kernel scheduler that never handles data, so $G_{\text{comp}} = \lceil W_2/(N_{\text{SM}} - 1) \rceil$. Communication differs most sharply. Comet splits Combine along the output dimension into N_{split} rounds, with $G_{\text{comm}} = N_{\text{split}}$. MegaMoE and FlashMoE fuse the return into the GEMM epilogue: each block directly pushes its completed tile, so communication is tile-granular and shares the computation round count. Thus, $G_{\text{comm}} = G_{\text{comp}} = \lceil W_2/N_{\text{SM}} \rceil$ for MegaMoE and $G_{\text{comm}} = G_{\text{comp}} = \lceil W_2/(N_{\text{SM}} - 1) \rceil$ for FlashMoE. UniEP instead returns each routed-token output through its communication SMs, yielding $G_{\text{comm}} = \lceil S_{R_{\text{a}}}^{\max}/N_{\text{SM}}^{\text{M}} \rceil$.

5.2 Analytical Model Refinement

Eq. (2) can be inaccurate when neither granularity divides the other: misaligned chunk boundaries may delay computation until the next communication round completes. In practice, we therefore compute stage time round by round. For Dispatch+GroupGEMM, starting with $T = 0$, we update

$$T \leftarrow \max\left(T, \left\lceil \frac{jG_{\text{comm}}}{G_{\text{comp}}} \right\rceil \frac{T_{\text{comm}}}{G_{\text{comm}}}\right) + \frac{T_{\text{comp}}}{G_{\text{comp}}}, \qquad (12)$$

for $j = 1, \ldots, G_{\text{comp}}$. Each computation round starts only after the previous round finishes and all required data arrives. The final T is the stage time. For GroupGEMM+Combine, we swap comp and comm.

6 Evaluation

6.1 Experiment Setup

6.1.1 Hardware Platforms. The experiments were conducted on four NVIDIA GPU hardware platforms, covering three generations of mainstream data center GPU architectures: Ampere, Hopper, and Blackwell. Table 5 lists the key parameters of each platform.

6.1.2 Kernel-Level Configurations. We extracted and de-duplicated (H, I, E, k) from the `config.json` files of all Hugging Face MoE checkpoints with at least 3,000 downloads as of May 1, 2026, yielding 103 shapes spanning $H \in [1024, 8192]$, $I \in [256, 5120]$, $E \in [8, 512]$, and $k \in [2, 12]$. These cover 7 LLM families, including DeepSeek-V3/V4 [8, 9], Qwen3 [24], GLM-4.x/5 [5], Kimi-K2 [13], Mixtral [12], gpt-oss [20], and Phi-3 [1].

We test each shape at $S_{R_{\text{in}}} \in \{32, 64, 128, 256, 512, 1\text{K}, 2\text{K}, 4\text{K}, 8\text{K}, 16\text{K}\}$, covering decode-scale batches through 8K-token chunked prefill. Of $1030 \times 20 = 20{,}600$ configuration–strategy–platform combinations, 13,734 (66.7%) are measurable. The rest violate implementation constraints (e.g. constraints on E, k or B_M divisibility). Feasibility is a property of the implementation, not a choice on our part: a strategy is never penalized on a configuration it cannot run.

Differences in supported precision, MLP structure, and platform require two experiment groups (Table 6). Before timing, we validate every strategy on all measured configurations against two serial methods: Torch (Group I) or DeepEP (Group II). Relative ℓ_2 errors remain below 10^{-2} for BF16 and 5×10^{-2} for FP8, consistent with accumulation-order and quantization effects. All speedups thus compare functionally equivalent implementations.

Figure 5 illustrates the proportion of test configurations in which each strategy performs best, highlighting distinct regions of advantage. Across all platforms, no single strategy achieves optimality in more than 80.6% of cases.

6.1.3 Data-Driven Model Configurations. We fit each strategy on a random 20% of measured samples (averaging 137 per strategy–hardware pair) by minimizing MAPE with DE, and evaluate on the remaining 80%. Five seeds control both data splitting and DE initialization. This small training fraction is deliberate: the 17–23 physically interpretable parameters per strategy (bandwidth utilization and compute

Table 5. Hardware platform specifications.

Platform	F (TFLOPs)		B_{HBM}	B_{NVL}	N_{SM}	D
	BF16	FP8	(GB/s)	(GB/s)		
A100	312	–	2039	600	108	2/4/8
H20	148	296	4022	900	78	2/4/8
H800	989	1979	3350	400	132	2/4/8
GB200	2500	5000	8000	1800	152	2/4

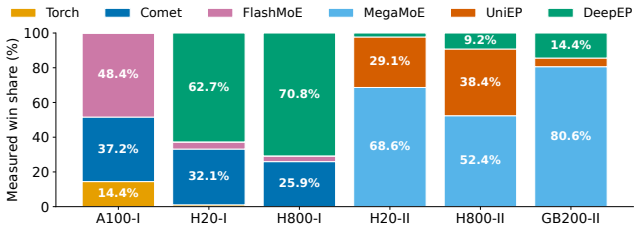

Figure 5. Performance win rates of MoE EP strategies across test configurations.

efficiency) are far fewer than the training samples, limiting overfitting risk, while the large remaining set tests generalization to unseen configurations.

Table 7 lists kernel decompositions and DE budgets. For kernels with ≤ 4, 5–9, and ≥ 10 parameters, we empirically set $(N_p, \text{maxiter})$ to $(15, 80)$, $(20, 120)$, and $(30, 200)$, respectively, where N_p is population size and maxiter is the iteration limit. All use mutation-factor dithering and $CR = 0.9$. Fitting all 90 instances across four GPU platforms takes about 13 minutes on an 8-vCPU Intel Xeon host (2.50 GHz, 16 GB memory), negligible compared with measurement collection.

6.1.4 E2E-Level Configurations.

We evaluate Group II (Table 6) on one fully NVLink-connected 8×H20 node using SGLang v0.5.15-post1 and vLLM 0.28.0, with CUDA 13.0 (nvcc 12.8), PyTorch 2.13.0, and NCCL 2.30.7. Both use DP attention (dp_size $= D$), EP $= D$, and TP $= 1$. (1) SGLang uses FlashInfer attention, Mooncake PD KV transfer, and a PD-aware router; (2) vLLM uses FlashAttention-3/FA-MLA, NIXL over NVLink, and an OpenAI-compatible proxy.

The three Group II strategies replace only the MoE layer and are selected via launch flags or environment variables. BF16 checkpoints are quantized to block-FP8 at load time; Qwen3-235B-A22B-FP8 uses native block-FP8. Each scenario starts a fresh server with overlap/async scheduling, prefix caching, and shared-expert fusion disabled; decode CUDA graphs enabled; an 8192-token per-rank prefill budget; and a GPU memory budget of 80% per device.

Both engines share the models and deployments in Table 8. SGLang replays the Mooncake production trace [23], and vLLM replays ShareGPT V3 [4] conversations, preserving natural length distributions under PD disaggregation. Colocated deployments use two subsets per trace: prefill-dominated and decode-dominated. We run $32D$ concurrent requests in total (32 per rank) and report the median of three runs per scenario after one warm-up.

Table 6. Two experiment groups to evaluate OverlapAdvisor.

	Group I	Group II
Precision	BF16	FP8
MLP	Standard, GELU	Gated, SwiGLU
Routing	Non-uniform, $\rho = 2.0$	Non-uniform, $\rho = 2.0$
Timing	CUDA Graph[a]	CUDA Graph[a]
A100	Comet, FlashMoE, Torch	–
H20, H800	Comet, FlashMoE, Torch, DeepEP-v1[b]	MegaMoE[c], UniEP, DeepEP-v1[b]
GB200	–	MegaMoE, UniEP, DeepEP-v2

[a]Comet and DeepEP-v1 Normal are timed with CUDA Event.

[b]DeepEP-v1 uses LowLatency (LL) for $S \leq 256$ and Normal otherwise, fitted separately but treated as one strategy by the selector.

[c]Cross-architecture ports: MegaMoE on Hopper follows the SM90 megakernel design of [28], and UniEP on Blackwell is ported from Hopper. Both preserve the original overlap structure.

6.1.5 Baselines.

We compare OverlapAdvisor with six fixed-strategy baselines (Table 6) at the kernel level and three of them (MegaMoE, UniEP, and DeepEP) end to end. Each baseline uses one strategy across configurations and, under PD disaggregation, for both prefill and decode.

6.1.6 Metrics.

Performance Improvement. Speedup over each baseline is $T_{\text{baseline}}/T_{\text{OverlapAdvisor}}$ for kernels and $Q_{\text{OverlapAdvisor}}/Q_{\text{baseline}}$ end to end, where T is execution time (μs) and Q is throughput (tokens/s); values above 1 indicate improvement. Kernel speedups are averaged over test configurations per seed; we report the mean and min–max range across five seeds.

Execution Time Regret. For each test configuration,

$$R_i = \left(\frac{T_i}{T_i^{\text{best}}} - 1 \right) \times 100\%, \tag{13}$$

where T_i^{best} is the fastest measured kernel execution time among feasible strategies. A CDF closer to the upper-left indicates lower regret across more configurations.

Prediction Error. We use mean absolute percentage error:

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \frac{|\widehat{T}_i - T_i|}{T_i}, \tag{14}$$

where $\widehat{T}_i$ and T_i are the predicted and measured execution times for test configuration i, and n is the test-set size per strategy–hardware pair and seed. We report mean MAPE and its min–max range across five seeds for each pair.

Selection Accuracy. Top-1 accuracy is the fraction of test configurations where OverlapAdvisor selects the fastest strategy, reported as the mean and min–max range across five seeds at each token batch size $S_{R_{\text{in}}}$.

6.2 Results and Analysis

6.2.1 Kernel-Level Performance.

Kernel Execution Time. Figures 6(a)–(b) compare OverlapAdvisor with six fixed-strategy baselines in execution-time speedup. (1) OverlapAdvisor outperforms all available fixed strategies, achieving 5.62× over the FlashMoE on GroupI-H800 and 1.86× over the DeepEP-v1 on GroupII-H20. (2) Against the strongest fixed strategy on each platform, OverlapAdvisor achieves 1.59× speedup over Comet on GroupI-A100; 1.15× and 1.14× over DeepEP-v1 on GroupI-H20 and

Table 7. Kernel decomposition and DE budget of each strategy. $|\theta|_{\mathrm{k}}$ denotes the number of fitted parameters per kernel, and $|\theta|_{\mathrm{tot}}$ denotes the total number per strategy.

Strategy	Partitioned Kernels	Kernels	$\lvert\theta\rvert_{\mathrm{k}}$	$(N_p, \mathrm{maxiter})$	$\lvert\theta\rvert_{\mathrm{tot}}$
Comet / UniEP	Dispatch+GroupGEMM / Act / GroupGEMM+Combine	3	3–7	(15, 80), (20, 120)	17
Torch	AG / permute / GroupGEMM / Act / GroupGEMM+unpermute+RS	5	3–10	(15, 80), (30, 200)	23
FlashMoE / MegaMoE	MegaKernel	1	17	(30, 200)	17
DeepEP-v1 Normal	Dispatch / permute / GroupGEMM / Act / GroupGEMM / unpermute / Combine	7	3–4	(15, 80)	23
DeepEP-v1 LL	Dispatch / GroupGEMM / Act / GroupGEMM / Combine	5	3–4	(15, 80)	17
DeepEP-v2	Dispatch / GroupGEMM+Act+GroupGEMM / Combine	3	3–11	(15, 80), (30, 200)	17

Table 8. Models and deployments used in E2E evaluation. PD a+b denotes a GPUs for prefill and b GPUs for decode.

ID	Model	$H/I/E/k$	EP degree (D)
E2E-1	Qwen3-30B-A3B	2048/768/128/8	PD 2+2 (2)
E2E-2	GLM-4.7-Flash	2048/1536/64/4	PD 2+2 (2)
E2E-3	Qwen3-Next-80B	2048/512/512/10	PD 4+4 (4)
E2E-4	Qwen3-235B-A22B-FP8	4096/1536/128/8	colocated (8)
E2E-5			

GroupI-H800; and 1.02×–1.03× over MegaMoE on the three Group II platforms. (3) Across both experiment groups, the average gain is 1.16×, peaking at 1.59×.

The modest 2–3% average gains over MegaMoE on Group II reflect many configurations where MegaMoE is already fastest and selection offers no gain. However, OverlapAdvisor outperforms MegaMoE by more than 5% on 107/773 (13.84%), 184/772 (23.83%), and 55/504 (10.91%) test configurations on GroupII-H20, GroupII-H800, and GroupII-GB200, respectively (Fig. 7). Thus, CSA avoids substantial performance losses in configurations where MegaMoE is suboptimal.

6.2.2 Across-Engine E2E-Level Performance. Across five deployments on two serving engines (Figure 8), OverlapAdvisor improves throughput over fixed-strategy baselines MegaMoE, UniEP, and DeepEP by averages of 1.78%, 4.57%, and 9.97%, respectively, with maxima of 9.28%, 9.95%, and 31.72%. These results demonstrate the necessity of strategy selection across different configurations and serving engines.

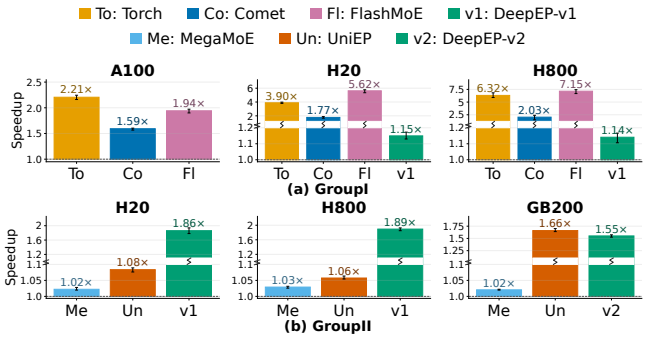

Figure 6. OverlapAdvisor kernel-level speedup over baselines.

6.3 Overhead

OverlapAdvisor incurs a one-time fitting cost of approximately 13 minutes per seed across all evaluated strategies and platforms on an 8-vCPU Intel Xeon host. At server launch, strategy selection takes approximately 6 μs without online profiling or warm-up. The performance-model code and fitted parameters for all 24 models together occupy approximately 26 KB of storage. The selected backend remains fixed during serving, requiring no profiling or runtime selection.

6.4 Ablation Study

6.4.1 Prediction Accuracy of UPP. Figure 9 reports UPP's MAPE across 20 ⟨strategy, platform⟩ pairs. DeepEP-v1's LL and Normal modes are fitted separately but treated as one strategy by CSA, yielding 24 models across 8 implementations of 6 strategies. Errors range from 3.3% to 15.9% (median

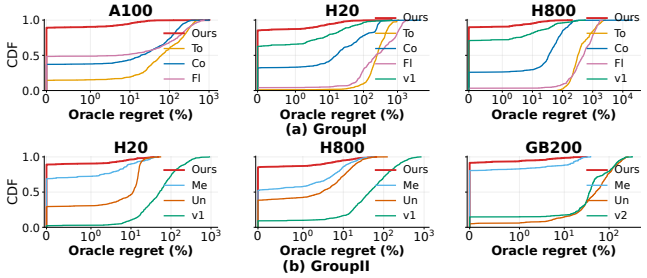

Figure 7. Kernel regret CDF of OverlapAdvisor and baselines.

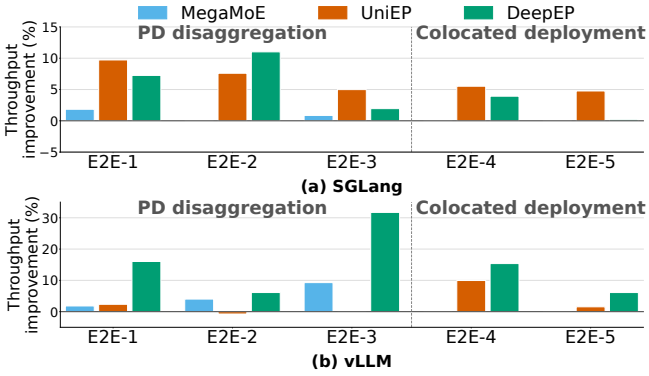

Figure 8. E2E-Level Evaluation: OverlapAdvisor throughput improvement over baselines (%).

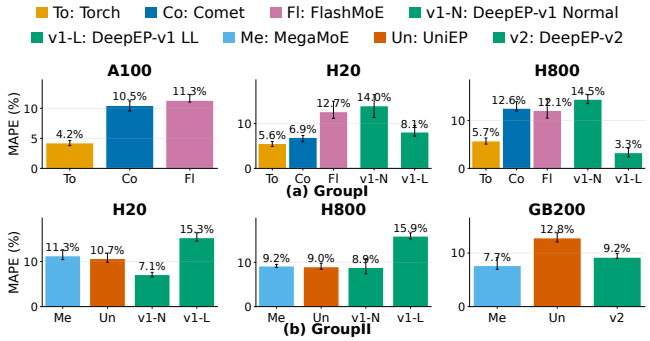

Figure 9. Ablation Study: Prediction MAPE of UPP.

9.9%), demonstrating UPP's effectiveness in modeling complex kernels such as FlashMoE and MegaMoE.

6.4.2 Predictor Comparison and Component Ablation of UPP. We compare UPP with learned predictors and ablate its analytical and data-driven models (Table 9), reporting mean MAPE across ⟨strategy, platform⟩ pairs under two data splits. The random split follows §6.1.3, training on 20% of the configurations. The extrapolation split trains on the four smallest token batches ($S_{R_{\text{in}}} \leq 256$, 40% of configurations) and tests on the six largest ($S_{R_{\text{in}}} \geq 512$). All fitted predictors use identical training and test data within each split. Fixed-efficiency variants use $\eta_{\text{gemm}} = \eta_{\text{mem}} = \eta_{\text{comm}} = 0.80$ and are evaluated on the same random-split test set.

Model Effectiveness. XGBoost and MLP use configurations, hardware specifications, and derived quantities V_{comm}, V_{mem}, and C_{gemm}, but not our granularity or efficiency functions. As Table 9(a) shows, OverlapAdvisor achieves 9.76% MAPE on the random split, versus 13.23% and 13.30% for XGBoost and MLP. Its advantage grows under extrapolation: 30.97% versus 53.85% and 49.90%. With limited training data, these predictors must learn execution time relationships from samples, whereas OverlapAdvisor explicitly models how volumes, granularities, and efficiencies compose.

Table 9. Prediction MAPE (%, lower is better). (a) OverlapAdvisor versus learned predictors, all fitted. (b) Ablation of analytical model and data-driven model.

(a) Model effectiveness

Predictor	Random split	Extrapolation
OverlapAdvisor	**9.76**	**30.97**
XGBoost	13.23	53.85
MLP	13.30	49.90

(b) Analytical and data-driven model effectiveness

	Random split		Extrapolation
Analytical model	Fixed	Fitted	Fitted
Granularity-aware (OverlapAdvisor)	**55.68**	**9.76**	**30.97**
Serialized	62.95	9.99	39.31
Perfect overlap	59.08	9.89	35.45

Analytical Model Effectiveness. We replace the analytical model (Eq. (2)) with serialized execution or perfect overlap, leaving the remaining model unchanged (Table 9(b)). With fixed efficiencies, OverlapAdvisor achieves 55.68% MAPE, below both alternatives (62.95% and 59.08%). After fitting, OverlapAdvisor retains an advantage of up to 0.23 percentage points on the random split. Although fitted efficiency functions partially compensate for analytical model errors, this compensation lacks a physical basis and generalizes poorly under extrapolation: OverlapAdvisor achieves 30.97% MAPE versus 39.31% and 35.45%, an advantage of up to 8.34 percentage points. Together, these three settings demonstrate the effectiveness of OverlapAdvisor's analytical overlap structure.

Data-Driven Model Effectiveness. On the same random-split test set, fitting reduces OverlapAdvisor's MAPE from 55.68% to 9.76% (Table 9(b)). This demonstrates the importance of platform- and strategy-specific parameter fitting.

6.4.3 Training Data Ratio of UPP. Following the protocol in Section 6.1.3, we vary the training ratio over 5%, 10%, 20%, 40%, 60%, and 80%, reporting average MAPE and strategy-selection accuracy. Figure 10 shows the mean and min–max range across five random seeds.

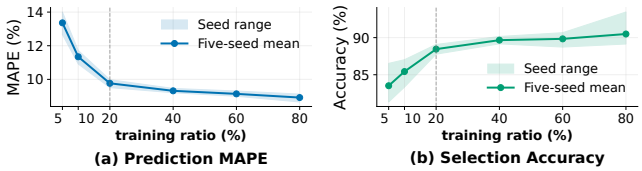

Figure 10. Training efficiency and accuracy of UPP.

As the training ratio increases from 5% to 20%, UPP's MAPE drops from 13.36% to 9.76%, and CSA's accuracy rises from 83.52% to 88.43%; both curves flatten thereafter. At 20%, each seed uses about 2.75K hardware–strategy samples and takes 13 minutes on an eight-core Intel Xeon 2.50 GHz CPU. Raising the ratio to 40% doubles profiling to 5.50K samples for marginal gains. We therefore select 20% using the elbow method to balance prediction accuracy, selection accuracy, and profiling overhead. OverlapAdvisor generalizes from this limited train set to the remaining 80% of configurations, supporting the effectiveness of UPP and CSA.

6.4.4 Selection Accuracy of CSA. Figure 11 shows top-1 selection accuracy versus token size. Accuracies are 89.2%, 85.4%, and 89.9% for GroupI-A100/H20/H800, and 89.2%, 85.3%, and 91.6% for GroupII-H20/H800/GB200, respectively, giving an average accuracy of 88.43% across both experiment groups.

Accuracy is generally lower when leading strategies have nearly identical performance, such as MegaMoE and UniEP at small token size on GroupII-H20 and GroupII-H800, or MegaMoE and DeepEP-v2 at large token size on GroupII-GB200. In these near-tie regimes, selecting the second-fastest

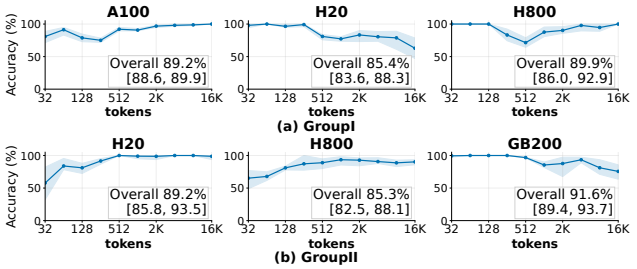

Figure 11. Ablation Study: Selection Accuracy of CSA.

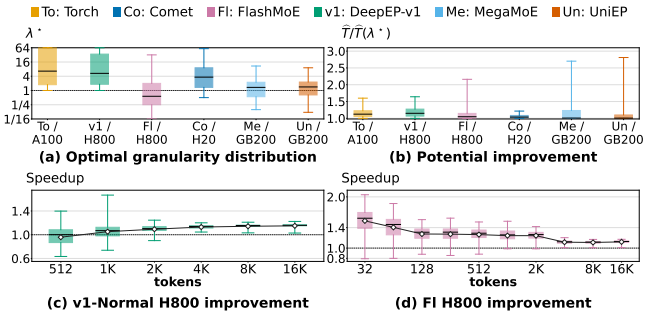

Figure 12. Optimal granularity distribution, potential improvement and realized improvement.

strategy incurs little penalty, so occasional top-1 errors have limited impact on the speedups and regret in Fig. 6 and Fig. 7.

6.5 Analysis and Optimization of Existing Strategies

Beyond strategy selection, OverlapAdvisor can diagnose bottlenecks related to granularity by assessing whether native granularities effectively balance overlap and efficiency, while identifying opportunities for tuning. With fitted efficiency parameters fixed, we scale native granularities as $G_j(\lambda) = \max(1, \lambda G_j^{\text{native}})$ and find the $\lambda^\star$ minimizing predicted execution time. Finer partitioning improves overlap but reduces per-round volume, potentially lowering communication and memory efficiency. Figures 12(a) and (b) show the distributions of $\lambda^\star$ and potential improvement relative to native granularities, respectively, revealing three regimes.

Coarse granularity limits overlap. Torch and DeepEP-v1 lack native overlap, with median $\lambda^\star$ values of 6.58 and 5.30 and median potential improvement of 1.123× and 1.149×, respectively. Comet supports overlap but remains too coarse: communication limits its two stages to $D-1$ ring steps and N_{split} combine rounds, preventing pipeline depth from scaling with workload. Refinement yields a median $\lambda^\star = 3.67$, but only 1.036× median headroom.

Guided by this diagnosis, we refine DeepEP-v1 by partitioning tokens into g groups and dispatching them in group-major order. We choose g from 2 to 8 depending on the token count (e.g., $g = 2$ for 512 tokens and $g = 8$ for $16K$ tokens) to balance computation efficiency and overlap. Per-group arrival counters allow each group's GroupGEMM to start as soon as its inputs arrive, overlapping computation with the remaining dispatch. This increases $G^{(1)}_{\text{comp}} = G^{(1)}_{\text{comm}}$ from 1 to g. As shown in Figure 12(c), the refinement achieves a median speedup of 1.099×, reaching 1.158× at $S_{R_{\text{in}}} = 16K$ and improving 511 of 594 configurations.

Excessive granularity reduces efficiency. Only FlashMoE has a median $\lambda^\star < 1$ (0.57), with a median predicted speedup of 1.047× from coarsening. On H800-I with $(S_{R_{\text{in}}}, H, I, E, k, D) = (4096, 4096, 2048, 128, 8, 8)$, $\rho = 2$, and $B_M = B_N = 128$, Dispatch+GroupGEMM splits the 131 non-scheduler SMs into 98 compute and 33 communication blocks, giving $G^{(1)}_{\text{comp}} = \lceil W_1/98 \rceil = 84$ and $G^{(1)}_{\text{comm}} = \lceil DE_{\text{L}}/33 \rceil = 4$. In GroupGEMM + Combine, each block computes and returns one output tile, yielding $G^{(2)}_{\text{comp}} = G^{(2)}_{\text{comm}} = \lceil W_2/131 \rceil = 126$.

We coarsen only second-stage communication by grouping each row block's $\lceil H/B_N^{(2)} \rceil = 32$ tiles under one signal. This increases the Combine payload by 32×, cuts signals by 32×, and reduces $G^{(2)}_{\text{comm}}$ from 126 to $\lceil E_{\text{L}} \lceil \rho S_E / B_M \rceil / 131 \rceil = 4$. Communication efficiency improves, but the overlap divisor $\min(G^{(2)}_{\text{comp}}, G^{(2)}_{\text{comm}})$ in Eq. (2) drops from 126 to 4. OverlapAdvisor identifies when the efficiency gain outweighs this overlap loss. Across 1030 measured configurations on 8×H800, coarsening achieves a median speedup of 1.217×, improving 910 configurations (Figure 12(d)).

Granularity is not always the bottleneck. MegaMoE and UniEP lie near flat optima: their median $\lambda^\star$ values are 1.35 and 1.41, but median potential improvements are only 1.017× and 1.015×. On GB200-II with $(S_{R_{\text{in}}}, H, I, E, k, D) = (8\text{K}, 2048, 512, 512, 10, 4)$, MegaMoE takes 2.328 ms versus DeepEP-v2's 1.862 ms despite much finer granularities. Its effective (GEMM, HBM, NVLink) efficiencies are $(0.576, 0.701, 0.598)$ versus $(0.763, 0.790, 0.804)$; granularity tuning alone cannot close this efficiency gap.

7 Related Work

We group related work into kernel modeling, overlap optimization, and strategy integration (Table 10).

Kernel modeling. Roofline [29] identifies compute and memory bottlenecks through arithmetic intensity and peak performance bounds. PipeWeave [32] combines analytical pipeline-cycle estimates with a lightweight MLP for cross-pipeline coupling. Both focus on individual kernels rather than cross-strategy EP overlap.

Overlap optimization. Lagom [30] uses a unified overlap loss to tune kernel communication parameters. Tutel [10] supports low-cost parallelism switching through an invariant expert-weight layout and tunes pipeline depth at the engine/runtime layer. Neither provides a unified performance model across diverse EP overlap implementations.

Strategy integration. DynaFlow [21] decouples model definitions from execution schedules to integrate intra-device parallelism strategies, but provides no predictive selector.

OverlapAdvisor instead provides a unified EP overlap model for strategy selection and granularity diagnosis, validated across MoE workloads and hardware.

Table 10. Comparison of related work.

Work	Scope	Layer	Function
Roofline	Kernel	Kernel	Performance model
PipeWeave	Kernel	Kernel	Performance model
Lagom	Overlap	Kernel	Parameter tuning
Tutel	MoE overlap	Engine	Mode switching
DynaFlow	Overlap	Strategy	Strategy integration
OverlapAdvisor	**EP overlap**	**Strategy**	**Performance model, strategy adaptation and tuning**

8 Conclusion and Future Work

OverlapAdvisor unifies performance modeling for MoE EP strategies, enabling selection and optimization in two serving engines. First, it predicts performance with 3.3%–15.9% MAPE. Second, its launch-time selection takes about 6 μs without profiling, achieving 88.43% top-1 accuracy. Third, it delivers 1.16× average and 1.59× maximum kernel speedups over the best fixed strategies. Fourth, it improves end-to-end throughput over MegaMoE, UniEP, and DeepEP by averages of 1.78%, 4.57%, and 9.97%, with gains up to 31.72%.

Future work will extend OverlapAdvisor to multi-node deployments, model SM- and warp-level resource contention among overlapping kernels, and further unify prediction across platforms and strategies without separate fitting.

References

[1] Marah Abdin et al. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219 [cs.CL] https://arxiv.org/abs/2404.14219

[2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. arXiv:2308.16369 [cs.LG] https://arxiv.org/abs/2308.16369

[3] Osayamen Jonathan Aimuyo, Byungsoo Oh, and Rachee Singh. 2025. FlashMoE: Fast Distributed MoE in a Single Kernel. arXiv:2506.04667 [cs.DC] https://arxiv.org/abs/2506.04667

[4] anon8231489123. 2023. ShareGPT Vicuna Unfiltered. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered.

[5] Aohan Zeng and others. 2026. GLM-5: from Vibe Coding to Agentic Engineering. arXiv:2602.15763 [cs.LG] https://arxiv.org/abs/2602.15763

[6] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Chengji Yao, Ziheng Jiang, Haibin Lin, Xin Jin, and Xin Liu. 2024. FLUX: Fast Software-based Communication Overlap On GPUs Through Kernel Fusion. arXiv:2406.06858 [cs.LG] https://arxiv.org/abs/2406.06858

[7] Minyu Cui and Miquel Pericas. 2026. Resource-aware Computation-Communication Overlap for multi-GPU ML Workloads. arXiv:2606.09200 [cs.DC] https://arxiv.org/abs/2606.09200

[8] DeepSeek-AI. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[9] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348 [cs.CL] https://arxiv.org/abs/2606.19348

[10] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, Joe Chau, Peng Cheng, Fan Yang, Mao Yang, and Yongqiang Xiong. 2023. Tutel: Adaptive Mixture-of-Experts at Scale. arXiv:2206.03382 [cs.DC] https://arxiv.org/abs/2206.03382

[11] JianboDong. 2025. Add imbalance factor in test_low_latency. DeepEP Pull Request #393. https://github.com/deepseek-ai/DeepEP/pull/393.

[12] Albert Q. Jiang et al. 2024. Mixtral of Experts. arXiv:2401.04088 [cs.LG] https://arxiv.org/abs/2401.04088

[13] Kimi Team. 2026. Kimi K2: Open Agentic Intelligence. arXiv:2507.20534 [cs.LG] https://arxiv.org/abs/2507.20534

[14] Diederik P. Kingma and Jimmy Ba. 2017. Adam: A Method for Stochastic Optimization. arXiv:1412.6980 [cs.LG] https://arxiv.org/abs/1412.6980

[15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180 [cs.LG] https://arxiv.org/abs/2309.06180

[16] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. arXiv:2006.16668 [cs.CL] https://arxiv.org/abs/2006.16668

[17] Wanqian Li, Jintao Peng, Zongfei Jing, Tianyu Zhang, Ze Long, Xianjie Qiao, Xiaoming Chen, Dongxu Yang, Kefeng Duan, and June Yang. 2026. DWDP: Distributed Weight Data Parallelism for High-Performance LLM Inference on NVL72. arXiv:2604.01621 [cs.DC] https://arxiv.org/abs/2604.01621

[18] Dong C. Liu and Jorge Nocedal. 1989. On the limited memory BFGS method for large scale optimization. *Mathematical Programming* 45 (1989), 503–528. https://api.semanticscholar.org/CorpusID:5681609

[19] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. arXiv:2104.04473 [cs.CL] https://arxiv.org/abs/2104.04473

[20] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] https://arxiv.org/abs/2508.10925

[21] Yi Pan, Yile Gu, Jinbin Luo, Yibo Wu, Ziren Wang, Hongtao Zhang, Ziyi Xu, Shengkai Lin, Baris Kasikci, and Stephanie Wang. 2026. DynaFlow: Transparent and Flexible Intra-Device Parallelism via Programmable Operator Scheduling. arXiv:2605.21603 [cs.DC] https://arxiv.org/abs/2605.21603

[22] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. arXiv:1912.01703 [cs.LG] https://arxiv.org/abs/1912.01703

[23] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170. https://www.usenix.org/conference/fast25/presentation/qin

[24] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[25] Rainer Storn and Kenneth V. Price. 1997. Differential Evolution – A Simple and Efficient Heuristic for global Optimization over Continuous Spaces. *Journal of Global Optimization* 11 (1997), 341–359. https://api.semanticscholar.org/CorpusID:5297867

[26] Stuart H. Sul, Simran Arora, Benjamin F. Spector, and Christopher Ré. 2025. ParallelKittens: Systematic and Practical Simplification of

Multi-GPU AI Kernels. arXiv:2511.13940 [cs.DC] https://arxiv.org/abs/2511.13940

[27] Octavian Alexandru Trifan, Karthik Sangaiah, Muhammad Awad, Muhammad Osama, Sumanth Gudaparthi, Alexandru Nicolau, Alexander Veidenbaum, and Ganesh Dasika. 2025. Eliminating Multi-GPU Performance Taxes: A Systems Approach to Efficient Distributed LLMs. arXiv:2511.02168 [cs.DC] https://arxiv.org/abs/2511.02168

[28] usernamehaha2022. 2026. DeepGEMM `add_hopper_mega` branch: SM90 MegaMoE megakernel optimizations. https://github.com/usernamehaha2022/DeepGEMM/tree/add_hopper_mega. Commit `6c3e048`; shared via the discussion of DeepGEMM PR #323, https://github.com/deepseek-ai/DeepGEMM/pull/323.

[29] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (April 2009), 65–76. doi:10.1145/1498765.1498785

[30] Guanbin Xu, ZhenGuo Xu, Yuzhe Li, Youhui Bai, Ping Gong, Chaoyi Ruan, and Cheng Li. 2026. Lagom: Unleashing the Power of Communication and Computation Overlapping for Distributed LLM Training. arXiv:2602.20656 [cs.DC] https://arxiv.org/abs/2602.20656

[31] Haoqi Yang, Luohe Shi, Qiwei Li, Zuchao Li, Ping Wang, Bo Du, Mengjia Shen, and Hai Zhao. 2025. Faster MoE LLM Inference for Extremely Large Models. arXiv:2505.03531 [cs.CL] https://arxiv.org/abs/2505.03531

[32] Kaixuan Zhang, Yunfan Cui, Shuhao Zhang, Chutong Ding, Shiyou Qian, Luping Wang, Jian Cao, Guangtao Xue, Cheng Huang, Guodong Yang, and Liping Zhang. 2026. PipeWeave: Synergizing Analytical and Learning Models for Unified GPU Performance Prediction. arXiv:2601.14910 [cs.PF] https://arxiv.org/abs/2601.14910

[33] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, Quan Chen, and Xin Liu. 2025. Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts. arXiv:2502.19811 [cs.DC] https://arxiv.org/abs/2502.19811

[34] Chenggang Zhao et al. 2026. [Public release 26/04] Introducing Mega MoE, FP4 Indexer and other features/fixes. https://github.com/deepseek-ai/DeepGEMM/pull/304. GitHub pull request #304, deepseek-ai/DeepGEMM.

[35] Chenggang Zhao, Zhean Xu, Liang Zhao, Jiashi Li, Chenhao Xu, Anyi Xu, Shengyu Liu, Kexing Zhou, and Kuai Yu. 2025. DeepGEMM: clean and efficient BLAS kernel library on GPU. https://github.com/deepseek-ai/DeepGEMM.

[36] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient expert-parallel communication library. https://github.com/deepseek-ai/DeepEP.

[37] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104 [cs.AI] https://arxiv.org/abs/2312.07104

[38] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, and Xin Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels using Tile-Centric Primitives. arXiv:2503.20313 [cs.DC] https://arxiv.org/abs/2503.20313

[39] Size Zheng, Xuegui Zheng, Li wen Chang, and Jidong Zhai. 2026. UniEP: Unified Expert-Parallel MoE MegaKernel for LLM Training. arXiv:2604.19241 [cs.DC] https://arxiv.org/abs/2604.19241

[40] Zhuoshan Zhou, Chen Zhang, Shuyi Zhang, Qijun Zhang, Haibo Wang, Zhe Zhou, Zhipeng Tu, Guangyu Sun, Yijia Diao, Zhigang Ji, Jingwen Leng, Guanghui He, and Minyi Guo. 2026. MoE-Hub: Taming Software Complexity for Seamless MoE Overlap with Hardware-Accelerated Communication on Multi-GPU Systems. arXiv:2605.05888 [cs.AR] https://arxiv.org/abs/2605.05888

[41] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Tian Tang, Qinyu Xu, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2025. NanoFlow: Towards Optimal Large Language Model Serving Throughput. arXiv:2408.12757 [cs.DC] https://arxiv.org/abs/2408.12757